Prendere il materiale del corso da GitHub

> **Da fare all'inizio, e poi una volta a settimana.** Il materiale del corso sta su GitHub e **cambia durante il semestre**: lezioni nuove, correzioni, esercizi. Due comandi bastano per averlo sempre aggiornato.
>
> **Non è materia d'esame.** È il modo di avere i file, come la guida alla VM.
>
> **Serve un terminale.** I comandi `cd`, `ls` e `pwd` e i percorsi sono quelli di `L02_shell.md`: se quella lezione è chiara, questa è solo un comando in più.

---

1. Due parole su git e GitHub

git è un programma che tiene la storia di una cartella: ogni modifica, chi l'ha fatta e quando. Serve a chi scrive il materiale — a te, per ora, serve la parte che lo scarica e lo aggiorna.

GitHub è un sito dove una cartella gestita da git può stare, in modo che altri la leggano. Il repository del corso è pubblico:

https://github.com/QmLHS/InfoCourse

Puoi leggerlo dal browser, ma dal terminale è meglio: un comando e hai tutto in locale, un altro e lo aggiorni.

2. Verificare che git ci sia

```
$ git --version
git version 2.39.5
```

Se risponde `command not found`, installalo con conda, che non chiede i permessi di amministratore:

```
$ conda install -c conda-forge git
```

3. Clonare: una volta sola

Clonare significa scaricare il repository con tutta la sua storia. Fallo nella tua home, non dentro una cartella di lavoro:

```
$ cd
$ git clone https://github.com/QmLHS/InfoCourse.git
$ cd InfoCourse
$ ls
01_AmbienteDiLavoro  02_ProgrammareInPython  03_FocusOnData  README.md
```

Da qui in avanti quella cartella **è** il materiale del corso:

Cartella	Cosa contiene
`01_AmbienteDiLavoro/`	informatica, calcolatore, sistema operativo, shell, Python
`02_ProgrammareInPython/`	Python di base
`03_FocusOnData/`	pandas e matplotlib

In ciascuna: i lucidi in PDF, la dispensa da leggere prima, gli esercizi con le soluzioni, il riepilogo e i dati in `data/`.

4. Aggiornare: una volta a settimana

```
$ cd ~/InfoCourse
$ git pull
```

Due risposte possibili. Se non c'è niente di nuovo:

```
Already up to date.
```

Se è arrivato del materiale, git ti dice **quali file** sono cambiati:

```
Updating a71e238..01d237a
Fast-forward
 01_AmbienteDiLavoro/L02_esercizi.md | 1 +
 1 file changed, 1 insertion(+)
```

È anche il modo più rapido di sapere *cosa* ho aggiunto dall'ultima volta.

5. La regola d'oro

> **Non lavorare dentro la cartella clonata.** Copia il file che ti serve nella tua cartella di lavoro, e lì fai quello che vuoi.

```
$ mkdir -p ~/informatica/esercizi
$ cp ~/InfoCourse/01_AmbienteDiLavoro/data/CacciaAlTesoro.zip ~/informatica/esercizi/
```

Il motivo è pratico: `git pull` porta la versione mia, e se tu hai modificato lo stesso file git **si ferma** invece di sovrascriverti il lavoro.

6. Se l'hai fatto comunque

Succede. Al `git pull` successivo vedrai questo:

```
error: Your local changes to the following files would be overwritten by merge:
    01_AmbienteDiLavoro/L02_esercizi.md
Please commit your changes or stash them before you merge.
Aborting
```

Non è rotto niente e non hai perso niente: git si è rifiutato di procedere *proprio* per non sovrascriverti. Per prima cosa guarda cosa hai toccato:

```
$ git status --short
 M 01_AmbienteDiLavoro/L02_esercizi.md
```

La `M` sta per *modified*. Poi decidi:

```
# le tue modifiche non ti servono: buttale e aggiorna
$ git restore .
$ git pull
```

```
# le tue modifiche ti servono: mettile da parte prima, in una tua cartella
$ cp 01_AmbienteDiLavoro/L02_esercizi.md ~/informatica/mie_note.md
$ git restore .
$ git pull
```

`git restore .` cancella le tue modifiche ai file del clone, e non si torna indietro. È giusto così — quei file non sono tuoi — ma per questo la copia va fatta prima.

7. Senza git, se proprio

Dal sito, bottone verde **Code** → **Download ZIP**. Funziona, ma:

- ogni aggiornamento è uno scaricamento da capo;
- nessuno ti dice cosa è cambiato;
- ti ritrovi `InfoCourse-main`, `InfoCourse-main (1)`, `InfoCourse-main (2)` — esattamente il problema che git esiste per evitare.

Usalo come ripiego, non come abitudine.

8. Cosa non ti serve

Il repository è **pubblico**: per leggerlo e aggiornarlo non serve un account GitHub, non servono chiavi SSH né token, e non esiste un `git push` da fare. I comandi del corso sono quattro, e due li usi una volta sola.

Comando	Quando
`git --version`	una volta, per controllare che ci sia
`git clone <url>`	una volta sola, all'inizio
`git pull`	ogni settimana, dentro `~/InfoCourse`
`git status --short` · `git restore .`	solo se hai modificato i file del clone

Se vuoi capire come si usa git sul **tuo** lavoro — versionare i tuoi script di analisi, tenerne la storia, recuperare una versione di due settimane fa — c'è `focus_Git.md`, facoltativo.

Due esercizi

1. **Clona e aggiorna.** Clona il repository nella tua home, entra nella cartella, elenca il contenuto e lancia `git pull`. Che risposta ti dà, e perché?

Mostra la soluzione

```
cd
git clone https://github.com/QmLHS/InfoCourse.git
cd InfoCourse
ls
git pull            # Already up to date. — hai appena clonato, sei aggiornato
```

2. **Rimetti a posto un file.** Dentro il clone, aggiungi una riga a un qualunque `.md` con `echo "prova" >> nomefile.md`. Poi verifica con `git status` che git l'abbia notato, e riporta il file come era.

Mostra la soluzione

```
cd ~/InfoCourse/01_AmbienteDiLavoro
echo "prova" >> L02_riepilogo.md
git status --short          #  M 01_AmbienteDiLavoro/L02_riepilogo.md
git restore L02_riepilogo.md
git status --short          # nessuna riga: il file è tornato come nel repository
```

`git restore .` fa la stessa cosa su tutti i file modificati in un colpo.